

DataForge 사용 가이드

DT는 Excel에서 작성하고, DA와 Config는 기존 Unreal 원천을 읽습니다. 모든 작업의 진입점은 DataForge입니다.

DT | 작성 · 빌드

원천: XLSX / XLSM

대표 항목: StatGrowth

Refresh → Build Selected

→ Bake + Verify

실제 UDataTable와 Server JSON 생성

DA | 조회 · 추출

원천: UDataAsset / PrimaryDataAsset

대표 항목: DefaultInput

DA 탭 → 대상 선택

→ Extract Selected

UE가 읽은 property와 descriptor 조회

Config | 조회 · 추출

원천: Unreal Config .ini

대표 항목: ProjectSettings

Config 탭 → 대상 선택

→ Extract Selected

UE 기본값·병합 정책이 적용된 값 조회

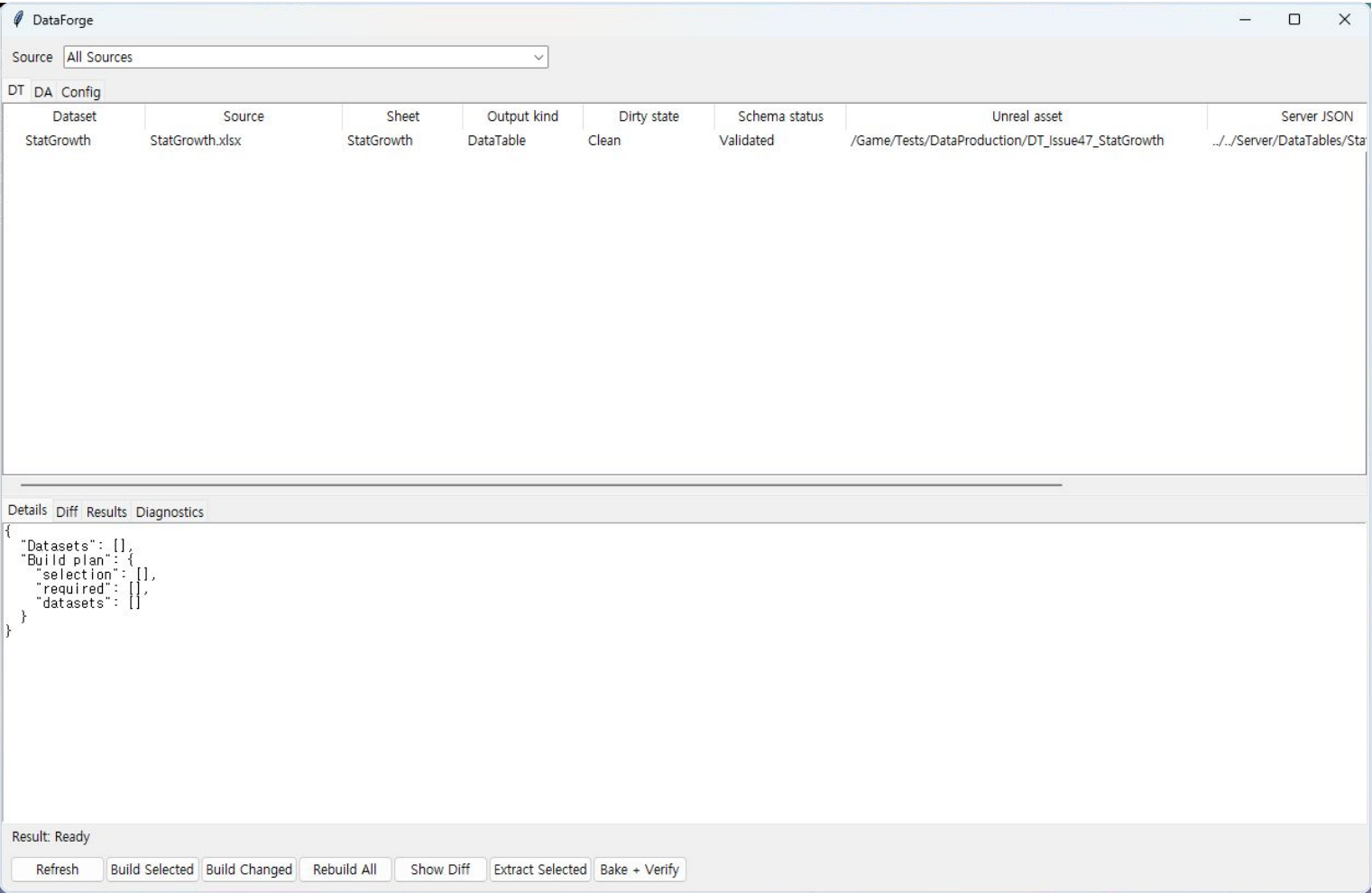
처음 사용하는 순서

1. 실행 준비(7쪽)
2. 화면과 버튼 확인(2쪽)
3. DT 빌드(3쪽) 또는 DA·Config 추출(4쪽)
4. 실제 산출물 확인(5쪽)
5. 상태·오류 대응(6쪽)

V1에서는 DT만 Bake합니다. DA·Config write-back과 Excel 전환은 지원 범위 밖입니다.

화면 읽기

상단 DT / DA / Config 탭으로 종류를 선택합니다. 각 탭의 dataset 선택은 탭을 바뀌도 유지됩니다.



① 종류 탭 · Source

DT / DA / Config 분류.
Source 필터는 현재 탭 안에서 적용.

② Dataset 목록

원천·대상·상태·schema 확인.
긴 컬럼은 가로 스크롤로 조회.

③ Inspector

Details: 선택 항목과 데이터
Diff: DT semantic 변경
Results / Diagnostics: 실행·경고

④ 작업 버튼

DT: Build Selected, Build Changed,
Rebuild All, Bake + Verify
DA·Config: Extract Selected

DT: Excel에서 실제 UDataTable까지

예제: StatGrowth.xlsx의 StatGrowth sheet. RowName은 안정적인 key이며 대소문자만 다른 중복도 허용하지 않습니다.

1 원천 편집

Data/StatGrowth/
StatGrowth.xlsx
값을 편집하고 저장합니다.
수식은 저장된 계산값 사용.

2 Refresh · Diff

DT 탭에서 Refresh.
대상을 선택하고 Show Diff.
변경 내용·상태·오류를
먼저 확인합니다.

3 Build

Build Selected: 선택 대상
Build Changed: Dirty 대상
필수 dependency를 포함.
아직 pending artifact 단계.

4 Bake + Verify

실제 UDataTable import.
별도 UE process readback.
Client/Server 값 비교 PASS
후 최종 산출물 게시.

버튼 선택 기준

Build Selected	선택한 DT와 필수 prerequisite만 빌드
Build Changed	Dirty DT와 영향받는 dependency만 빌드
Rebuild All	명시적인 전체 DT 재빌드. 평소 갱신에는 Build Changed 사용
Bake + Verify	가장 최근 Build의 pending DT artifact를 Bake·검증

Excel을 바꾼 뒤 기존 pending 결과를 그대로 Bake하면 verify가 변경을 감지해 거부합니다. 다시 Build하세요.

DA · Config: 원천을 유지하며 읽기

Unreal 실행 경로가 설정된 GUI에서 Extract Selected 버튼을 사용합니다. Commandlet 호출은 DataForge가 내부에서 처리합니다.

DA 탭

1. DefaultInput을 선택합니다.
2. Extract Selected를 누릅니다.
3. Details / Results에서 class, properties, descriptor, primaryAssetId를 확인합니다.

실제 예: SFInputConfig:DA_Input_Default
입력 Action, MappingContext, GameplayTag 조회.
Data Validation은 NotValidated로 관측됐습니다.

Config 탭

1. ProjectSettings를 선택합니다.
2. Extract Selected를 누릅니다.
3. Details / Results에서 category, section, entries를 확인합니다.

실제 예: Game / GeneralProjectSettings
UE 기본 ProjectVersion=1.0.0.0도 포함됩니다.
INI key 반복·순서는 UE 결과 그대로 보존합니다.

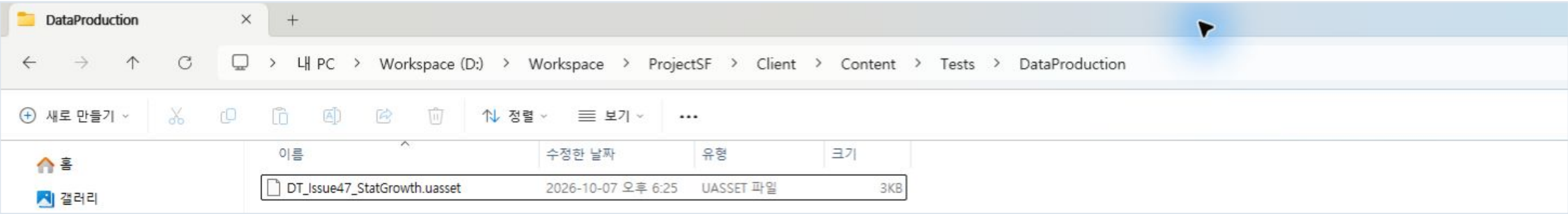
NeedsExtract라면

native cache가 없거나 원천이 바뀐 상태입니다. 해당 항목만 다시 Extract Selected 하세요.
Refresh는 원천·UAsset·Server JSON을 바꾸지 않습니다. DA·Config를 Build로 선택하면 거부됩니다.

빌드 결과 파일 확인

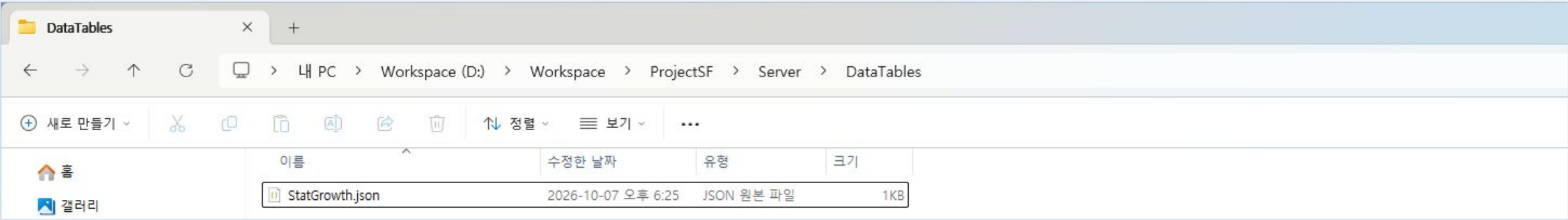
아래는 실제 파일 탐색기 캡처의 상단 파일 목록입니다. UAsset 내용 검증은 UE readback을 통해 수행했습니다.

UE 최종 산출물



Client/Content/Tests/DataProduction/DT_Issue47_StatGrowth.uasset (2,351 bytes)

Server 최종 산출물



Server/DataTables/StatGrowth.json (220 bytes)

실제 값: HR_000001 | Level 1 | MaxHP 100 | Attack 10 | Defense 5 | ElementalMastery 1

상태와 오류 대응

Clean은 마지막 결과와 의미가 같다는 뜻입니다. native Clean은 현재 원천에 대응하는 읽기 cache가 있다는 뜻입니다.

Clean / NO_CHANGES	변경 없음. Build Changed는 산출물을 다시 쓰지 않음.
New / Modified / SchemaChanged	신규 또는 의미-schema 변경. Diff 확인 후 DT Build.
DependencyDirty	의존 데이터 변경의 영향. 추가 required 대상 확인.
NeedsExtract	DA-Config 원천 변경 또는 cache 없음. 선택 추출.
Removed	manifest에서 제거됨. Refresh만으로 기존 산출물 삭제 안 함.

Invalid source-identity-schema 오류. Diagnostics 확인 후 수정.

INFO

source에 없는 native property는 안전한 UE default 사용 가능.

WARNING

비치명 Schema/C++ mismatch, 사용하지 않는 source field 등.

ERROR

실제 deserialize/import/save 실패. 기존 정상 산출물은 runner가 복원.

더블클릭 실행과 CLI 참조

제작자는 Tools/DataForge/Release/DataForge.exe를 실행합니다. Python 설치가 필요 없습니다.

첫 실행: manifest / 출력 폴더 / Unreal project / UnrealEditor-Cmd 경로 확인 → Open DataForge.
설정은 자동 저장됩니다. 이후 바로 GUI가 열리며 Settings에서 경로를 변경합니다.

```
python -m pip install -r Tools/DataForge/requirements.txt
$forge = "Tools/DataForge/dataforge.py"
$out = "D:/Workspace/CodexWorkSpace/issue47-dataforge/build"
$ue = "D:/UnrealEngine/UnrealEngine/Engine/Binaries/Win64/UnrealEditor-Cmd.exe"
$project = "D:/Workspace/ProjectSF/Client/SF.uproject"
python $forge gui --output $out --editor $ue --project $project
python $forge build --dataset StatGrowth --output $out
python $forge bake --output $out --editor $ue --project $project
python $forge extract --dataset DefaultInput ProjectSettings --output $out --editor $ue --project $project
```

위 명령은 개발자 CLI 참조입니다. 제작자는 EXE의 GUI 버튼을 사용합니다.

GUI에서 Bake는 가장 최근 pending Build를 처리합니다. 원천 유형과 작업 목적을 먼저 확인하세요.

검증 기준: 실제 DT E2E 및 별도 UE readback-Server parity PASS, 회귀 25개 PASS.

현재 범위: scalar StatGrowth DT, 명시적 manifest DA-Config 조회. 고급 DT type/reference,

USTRUCT 템플릿-legacy migration 확장, Server runtime package/load는 아직 후속 항목입니다.

상세 원본: Tools/DataForge/README.md · Docs/Technical/Architecture/DataProductionArchitecture.md
Production/Evidence/Issue-47/ValidationReport.json · GitHub Issue #47